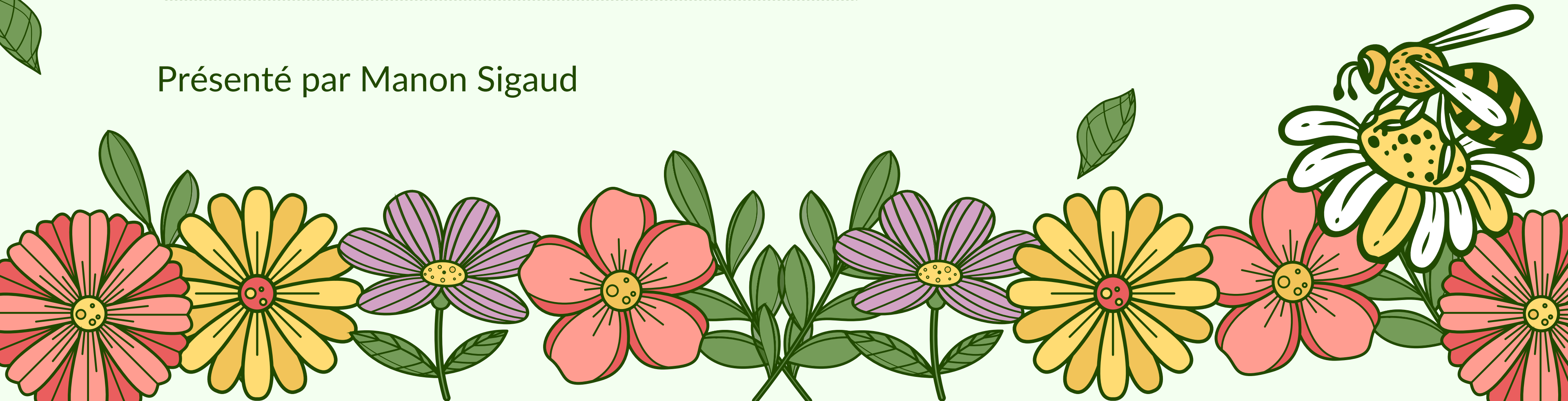


Le miel et les abeilles

Optimiser le trajet d'une abeille
avec un algorithme génétique

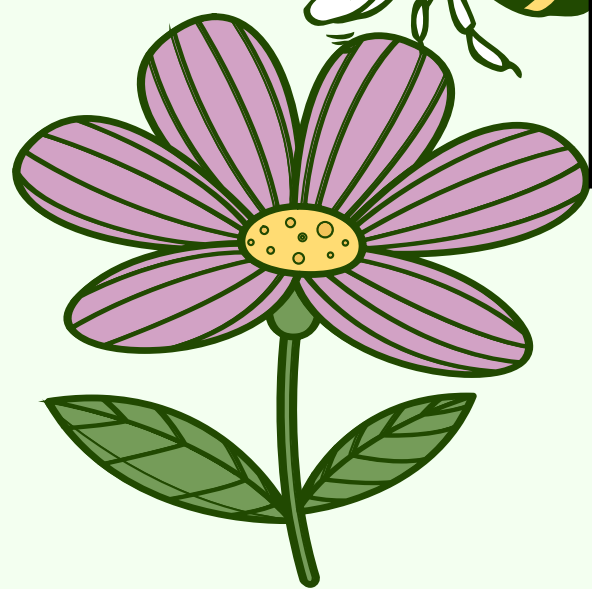
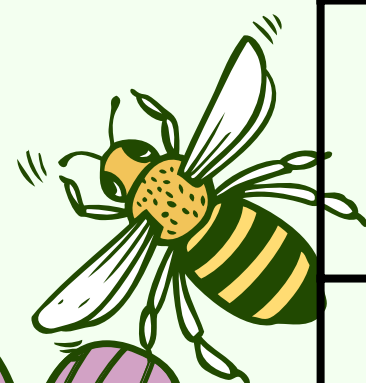
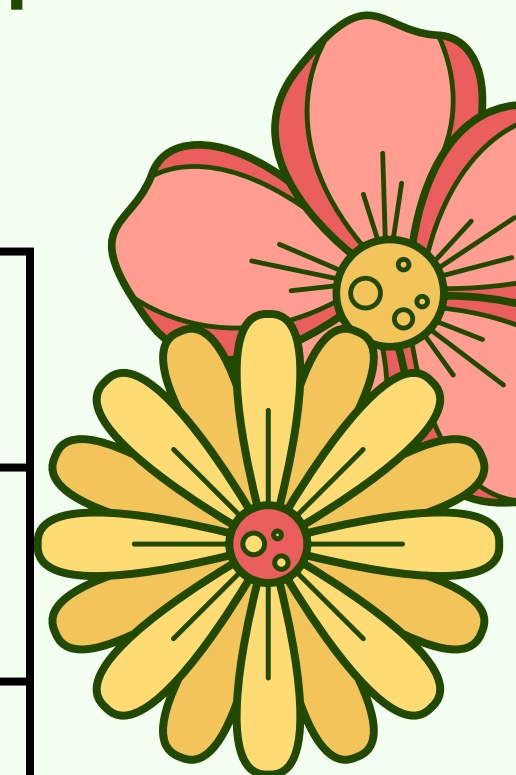
Présenté par Manon Sigaud





L'idée : imiter la nature grâce à un algorithme euristique

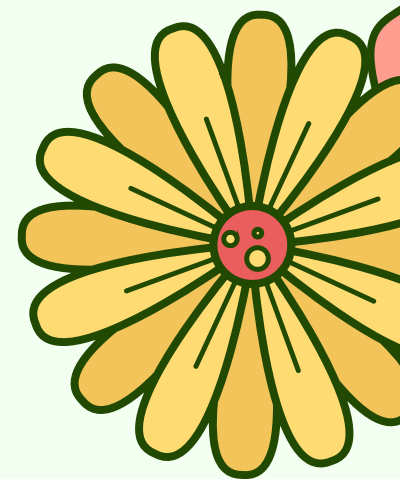
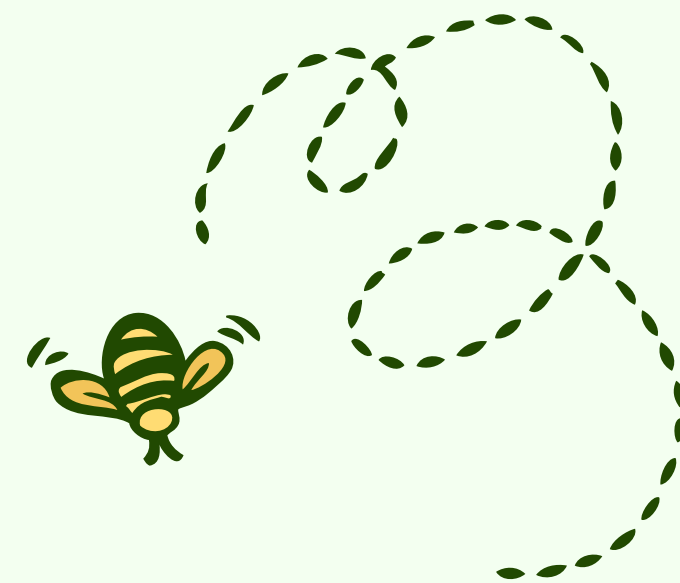
Nature	Mon algorithme	Dans ce projet
Individu	Une solution	Une abeille
ADN/Génôme	Encodage	L'ordre de visite des fleurs
Aptitude	Qualité	La distance (plus court = mieux)
Génération	Itération	Un cycle de reproduction





Étape 1 : 101 abeilles

Je crée 101 abeilles au hasard, chacune avec un trajet/ordre de visite aléatoire



Étape 2 : Mesure de la distance

Je mesure la distance du trajet de chaque abeille



Pythagore

Pour mesurer la distance entre 2 trajets, j'utilise le théorème de Pythagore, que j'additionne tout au long du parcours, c'est mon fitness.

$$d(A,B) = \sqrt{((x_B - x_A)^2 + (y_B - y_A)^2)}$$

$$L = d(\text{Ruche} \rightarrow \text{fleur 1}) + d(\text{fleur 1} \rightarrow \text{fleur 2}) + \dots + d(\text{fleur 50} \rightarrow \text{Ruche})$$



Étape 3 : La sélection

1. Je trie les abeilles de la meilleure à la pire (par distance)
2. Je calcule le nombre de parents : $101 \times 0.5 = 50$
3. Je garde les 50 meilleures abeilles, elles deviennent donc parents
4. La meilleure est toujours conservée (élitisme)
5. Les 51 places restantes seront occupées par leurs enfants



Étape 4 & 5 : Le croisement et la mutation



Parent 1 : [A B | C D E | F G]

Parent 2 : [D E G A F B C]

└─ je copie ─┐

Enfant : [A F B C D E G]

Deux parents et un enfant qui mélange leurs trajets. Je garde 1 morceau du parent 1 et je complète avec les fleurs du parent 2 (pas prises)

Croisement : Chaque fleur une seule fois

Avant : [A F B C D E G]

↕ on échange deux fleurs

Après : [A D B C F E G]

Pour ne pas tourner en rond, j'échange 2 fleurs au hasard dans le trajet, ça permet de sortir des impasses.

Mutation : Probabilité de 10% par enfant ≈ 5 mutations par génération

Techniques utilisées

Jupyter Notebook

Démarche exploratoire :
développer et visualiser



Matplotlib

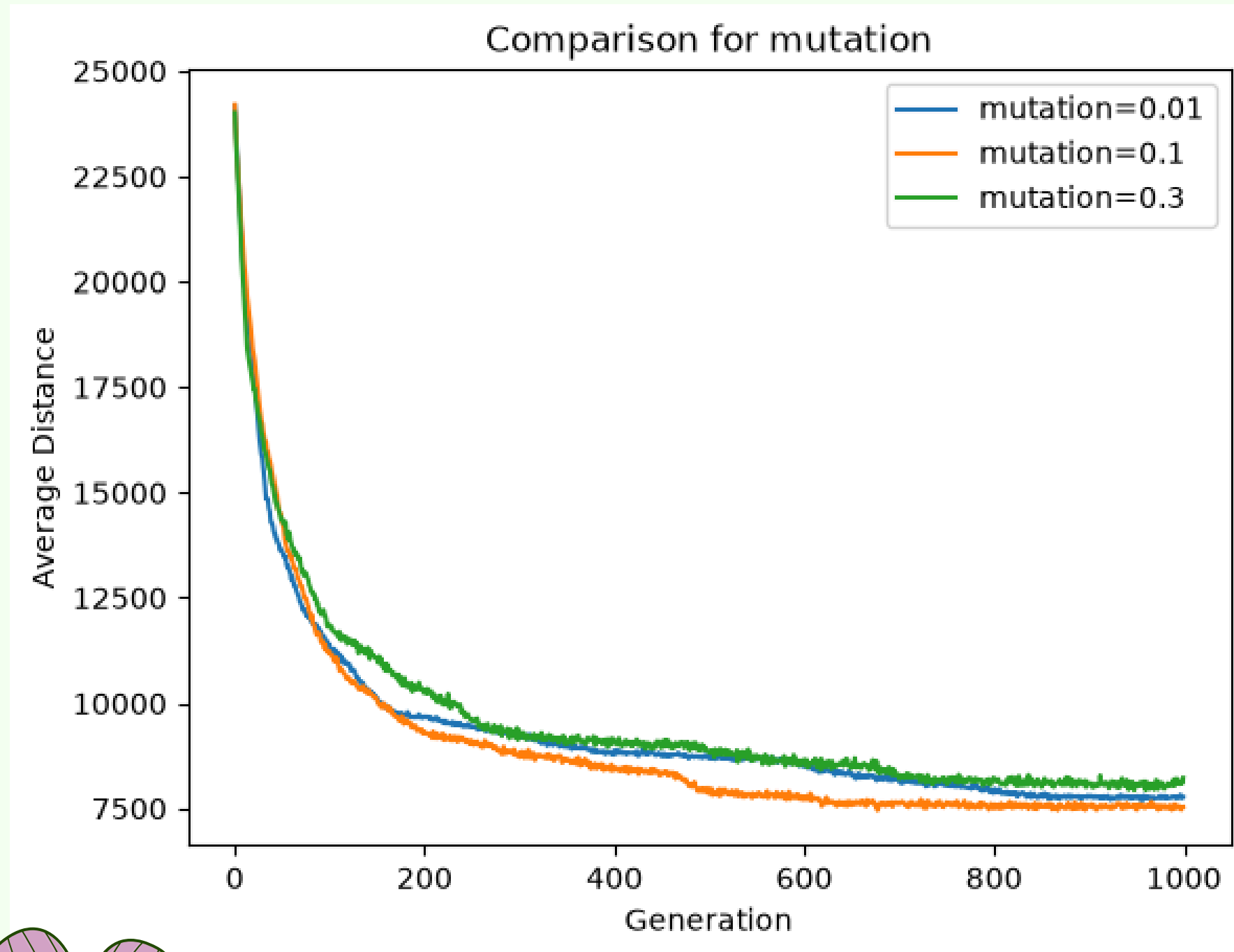
Courbes de convergence,
trajets



Networkx

Construire et dessiner

Taux de mutation

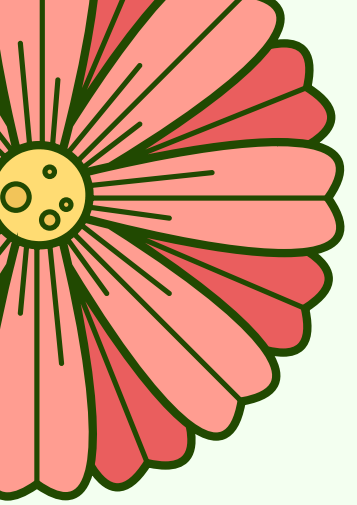


Test de 3 taux de mutation (1%, 10%, 30%):

1% : ça stagne (pas assez d'exploration)

10% : elle descend vite et bas, meilleur compromis entre explorer de nouvelles pistes et exploiter celles qu'on a.

30% : beaucoup de bruit mais aucun gain.



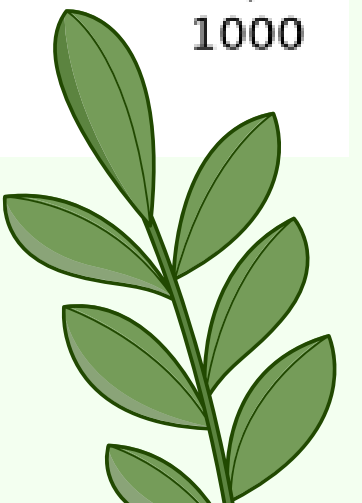
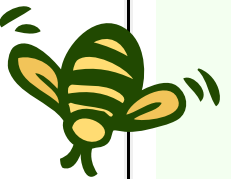
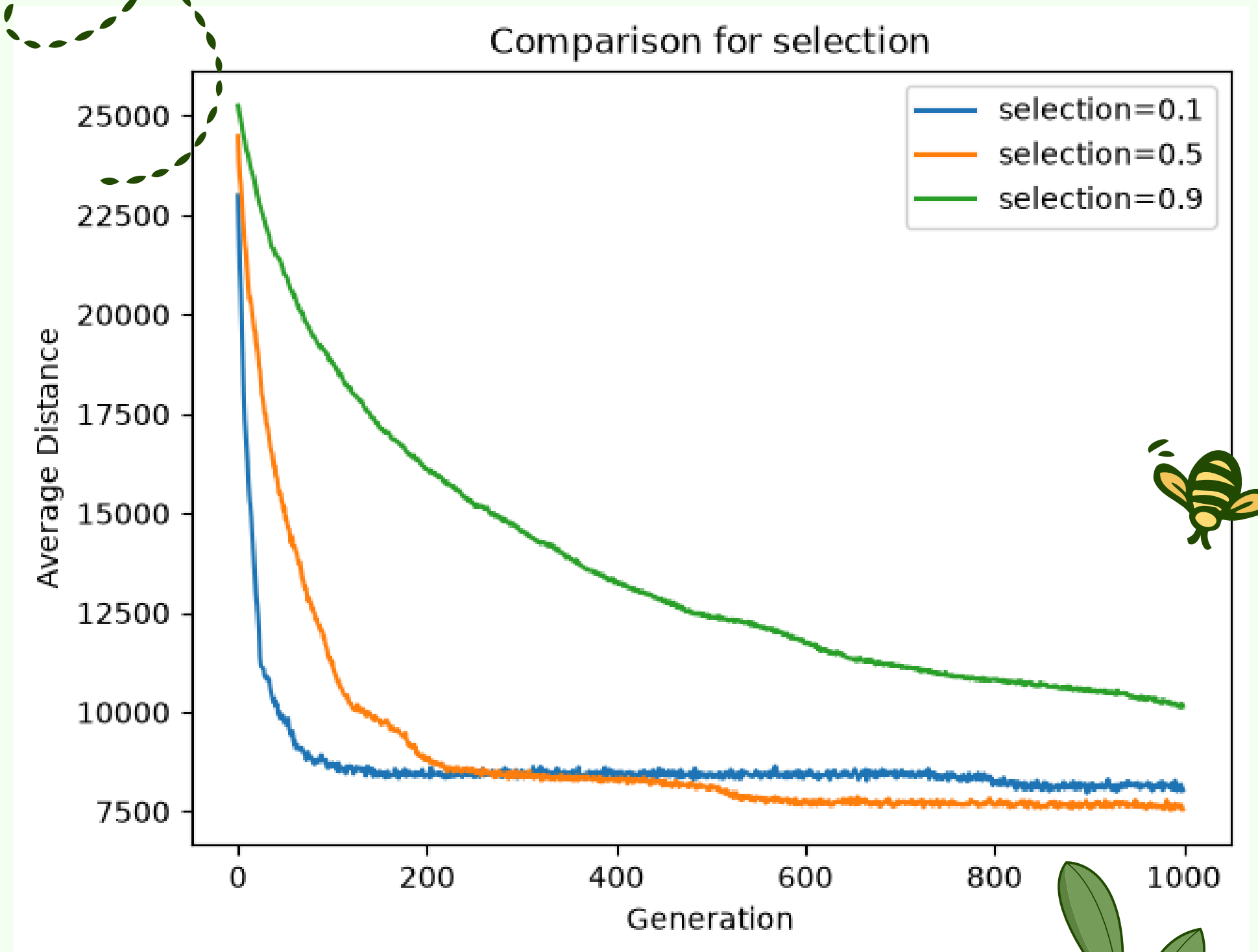
Taux de sélection

3 taux de sélection (10%, 50%, 90%) :

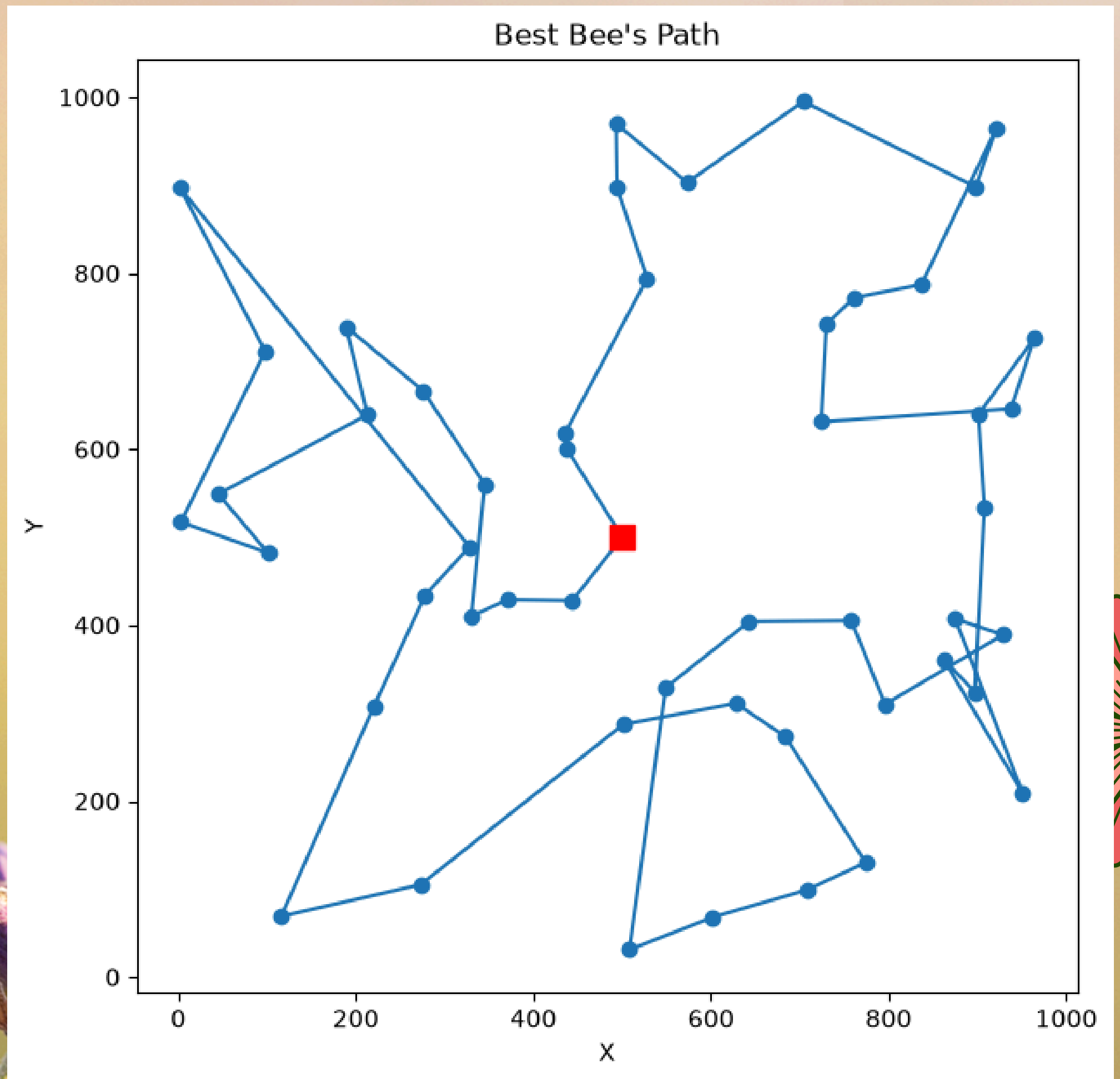
10% : ça descend vite mais stagne trop haut

50% : ça descend vite et bas, ni trop rigide, ni trop permissif (comme en ML pour régler un modèle)

90% : trop lent

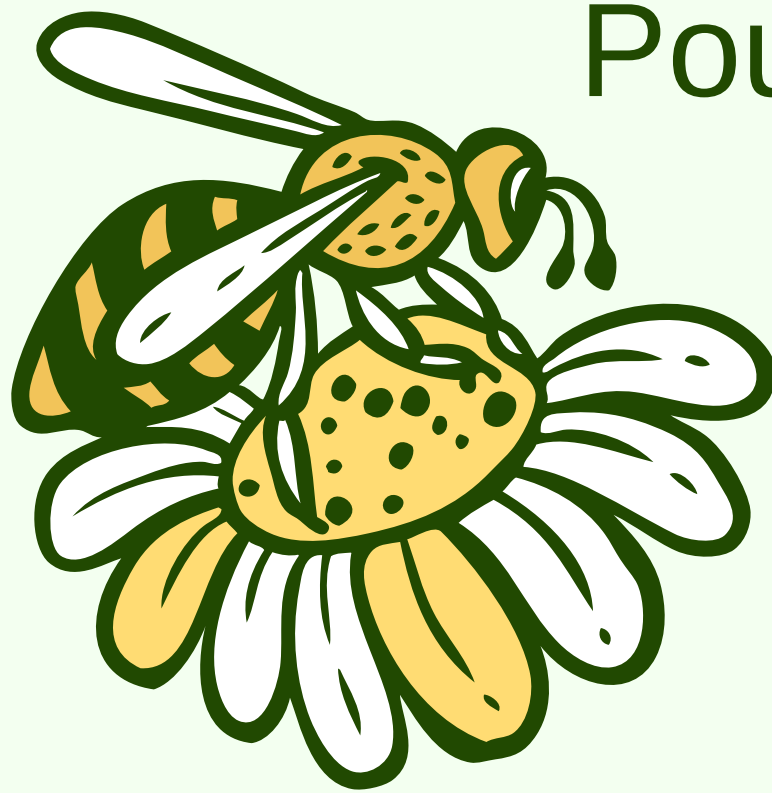


**Distance moyenne au
départ : 24400
Maintenant : 7700 en 1000
générations
Meilleur trajet trouvé :
7200**



Conclusion

Pourquoi c'est utile en data science ?



Optimisation : entraîner un modèle = minimiser une erreur
Algorithme génétique = optimisation sans gradient (quand on ne peut pas dériver)
J'ai appris à définir une fonction objectif, arbitrer exploration/exploitation, régler des paramètres intelligemment et j'ai compris que ça fonctionne pareil pour régler un modèle de ML.



Place à la
démo !

